

Clase 16 – Responsive Design

IIC2182 – Interfaces y Experiencia de Usuario



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Primer Semestre 2026

Agenda

De los fundamentos a los patrones canónicos, terminando con el error más común.

Fundamentos

-  Paisaje multi-dispositivo
-  Mobile-first vs Desktop-first
-  3 dimensiones de adaptación

Herramientas técnicas

-  Media queries y breakpoints
-  Fluid units y container queries
-  Imágenes responsive

Patrones de layout

-  Los 4 patrones canónicos
-  Touch y hide/show
-  Accesibilidad (prefers-*)

El antipatrón

-  Content Dispersion (NN/g)
-  4 costos de dispersar mal
-  Recomendaciones

Agenda

De los fundamentos a los patrones canónicos, terminando con el error más común.

Fundamentos

-  Paisaje multi-dispositivo
-  Mobile-first vs Desktop-first
-  3 dimensiones de adaptación

Herramientas técnicas

-  Media queries y breakpoints
-  Fluid units y container queries
-  Imágenes responsive

Patrones de layout

-  Los 4 patrones canónicos
-  Touch y hide/show
-  Accesibilidad (prefers-*)

El antipatrón

-  Content Dispersion (NN/g)
-  4 costos de dispersar mal
-  Recomendaciones

¿Qué es responsive design?

Una sola base de código que se adapta a múltiples contextos de uso.

“ Responsive Design

Diseño de interfaces que se adaptan dinámicamente al contexto en que se renderizan — viewport, dispositivo, capacidades — para entregar una experiencia óptima en cada uno.

⊗ Lo que NO es

No es una versión móvil separada (m.sitio.com).

No es escalar el desktop a un tamaño menor.

No es ocultar elementos en mobile y listo.



El término lo acuñó **Ethan Marcotte (2010)**: una misma página HTML/CSS que **responde** al medio en que se ve.

El paisaje multi-dispositivo

Diseñar para una sola pantalla ya no es una opción defendible.



~60% mobile

Más de la mitad del tráfico web global viene de teléfonos. En LATAM es aún más alto.



~40% desktop

Sigue siendo el contexto donde se completan tareas complejas: trabajo, compras de alto valor, investigación.



Tablet, TV, watch

Cada vez más superficies con web. Cada una con un viewport, un input y una atención distintos.



El usuario también puede **cambiar de dispositivo** durante una misma tarea — empieza en mobile, completa en desktop. La experiencia tiene que sentirse continua.

Las 3 dimensiones de la adaptación

Responsive no es solo "tamaño de pantalla". Hay tres ejes simultáneos.

Viewport

Ancho, alto, orientación.

De 320px (mobile estrecho) a 4K+ (monitores grandes).

Input

Touch, mouse, teclado, voz.

Hover no existe en touch. Tap targets necesitan más área.

Capacidades

Bandwidth, dark mode, reduced motion, dyslexia fonts.

Preferencias del sistema y accesibilidad.



Diseñar solo por **viewport** es responsive parcial. Diseñar por las tres dimensiones es responsive de verdad.

Mobile-first

Mobile-first

- 1 Diseñar para la pantalla más restrictiva primero.
- 2 Al expandir, agregar features y reorganizar para aprovechar el espacio.
- 3 CSS con `min-width` : reglas base son mobile, desktop las extiende.

Beneficio: obliga a priorizar lo esencial. Si una feature entra en mobile, es porque realmente importa.

Características del mobile first:

 **Performance:** bandwidth limitado de mobile fuerza optimización desde el inicio.

 **Prioridad:** la restricción de espacio es un filtro natural de jerarquía.

 **Convergencia:** Tailwind, Bootstrap 5+ y Material son mobile-first por default.

La intención original de Wroblewski (2009)

Mobile-first no significaba "diseñar pequeño y agrandar". Significaba priorizar.

“ *“La pantalla pequeña es una limitación creativa que fuerza a enfocarse en lo que de verdad le importa al usuario. Lo que sobra cuando estás obligado a caber, sobra siempre.”*

Luke Wroblewski — Mobile First (2009/2011)

✓ La filosofía correcta

Mobile como filtro de prioridades. Cada elemento gana su lugar. El layout para pantallas grandes se rediseña, no se agranda.

❗ La interpretación incorrecta

"Diseñé en mobile, ahora estiro a desktop". Resultado: pantallas grandes con elementos enormes, mucho whitespace, scroll infinito.

↑ Volveremos a este error al final.

Las herramientas técnicas

Cómo se implementa responsive en CSS

Media queries y breakpoints

El mecanismo base: aplicar reglas CSS solo cuando el contexto las cumple.

```
/* Base — mobile first */
.card {
  padding: 1rem;
  font-size: 14px;
}

/* A partir de 768px */
@media (min-width: 768px) {
  .card {
    padding: 2rem;
    font-size: 16px;
  }
}

/* A partir de 1280px */
@media (min-width: 1280px) {
  .card {
    padding: 3rem;
    font-size: 18px;
  }
}
```

Breakpoints estándar (Tailwind)

sm: ≥ 640px

md: ≥ 768px

lg: ≥ 1024px

xl: ≥ 1280px

2xl: ≥ 1536px

Mobile-first en práctica

<https://tailwindcss.com/docs/responsive-design>

Breakpoints por contenido, no por dispositivo

Un anti-patrón común: targetear modelos de teléfono específicos.

❌ Anti-patrón

Definir breakpoints según dispositivos del momento:

```
@media (max-width: 414px) { /* iPhone */ }  
@media (max-width: 768px) { /* iPad */ }
```

Quedan obsoletos cada año. Y dejan huecos entre tamaños comunes.

✓ Patrón

Definir breakpoints donde el layout se rompe:

1. Estiras el navegador desde 320px hacia arriba.
2. Donde el diseño se ve mal → ahí va el breakpoint.
3. El número exacto importa menos que la decisión visual.



Tus breakpoints deben servir **al contenido**, no a un catálogo de dispositivos que cambia cada año.

Fluid units: clamp()

En vez de saltos discretos, una progresión continua entre breakpoints.

```
/* Salto discreto (3 breakpoints) */
h1 { font-size: 1.5rem; }

@media (min-width: 768px) {
  h1 { font-size: 2rem; }
}

@media (min-width: 1280px) {
  h1 { font-size: 3rem; }
}

/* Fluid (una sola línea) */
h1 {
  font-size: clamp(1.5rem, 2vw + 1rem, 3rem);
  /*           ↑min      ↑preferred  ↑max */
}
```

En Tailwind (arbitrary values)

```
<h1 class="text-[clamp(1.5rem,2vw+1rem,3rem)]">
  Título fluido
</h1>
```

💡 ¿Cómo funciona clamp()?

min: nunca más pequeño que esto.

preferred: tamaño ideal (puede usar vw).

max: nunca más grande que esto.

✓ Beneficio

El texto crece continuamente con el viewport. No hay saltos abruptos al cruzar un breakpoint.

💡 También para spacing

Sirve para padding , margin , gap . No solo tipografía.

Container queries

Un componente que se adapta a su contenedor, no al viewport global.

```
/* El padre declara que es un contenedor */
.sidebar {
  container-type: inline-size;
}

/* El hijo reacciona a SU contenedor */
@container (min-width: 400px) {
  .card {
    display: grid;
    grid-template-columns: auto 1fr;
  }
}
```

Misma tarjeta funciona en sidebar estrecho, en grid central, o en full-width sin saber dónde vive.

💡 ¿Por qué importa?

Con media queries, un componente "sabe" el tamaño del viewport. Pero un mismo componente puede vivir en un sidebar pequeño o en el contenido principal grande.

En Tailwind (@container)

```
<div class="@container">
  <div class="@md:grid @md:grid-cols-2 gap-4">
    <!-- 1 col por defecto, 2 cuando -->
    <!-- el contenedor ≥ 28rem (@md) -->
  </div>
</div>
```

✓ Soporte

Disponible en todos los browsers modernos desde 2023.

Imágenes responsive

No basta con `max-width: 100%`. La imagen correcta para cada contexto.

srcset — distintas resoluciones

```

```

El navegador elige la más adecuada según viewport y densidad de pantalla (retina).

picture — formato + art direction

```
<picture>  
  <!-- Formato moderno si el browser lo soporta -->  
  <source srcset="image.webp" type="image/webp">  
  
  <!-- Versión responsive para mobile -->  
  <source media="(max-width: 600px)"  
         srcset="img-small.jpg">  
  
  <!-- Fallback obligatorio -->  
    
</picture>
```

Distinto formato o recorte según browser y viewport.



`<img>` obligatorio: debe ser el último hijo. Sin él no se renderiza nada — y aporta el `alt`.



El orden importa: el browser usa el primer `<source>` que matchea. Pon las reglas más específicas primero.



Estilos al `<img>`: aplicar `width`, `object-fit`, etc. al `<img>` anidado, no al `<picture>`.



Agrega `loading="lazy"` a imágenes below-the-fold. Performance gratis, sin riesgo.

Patrones de layout responsive

Cuatro formas canónicas de reorganizar el layout

Los 4 patrones de Wroblewski

Cada patrón resuelve la pregunta "¿qué pasa con las columnas al cambiar de tamaño?" de forma distinta.

1. Mostly Fluid

El layout es elástico — las columnas se estiran. Solo en el breakpoint más bajo se reordenan en una sola columna.

El más común y simple.

2. Column Drop

Múltiples columnas en desktop. Al achicar, las columnas se van **apilando una bajo la otra**.

Buena para contenido editorial.

3. Layout Shifter

Layout completamente distinto en cada breakpoint — no es solo apilamiento, es reorganización.

Más flexible, pero más trabajo.

4. Off-Canvas

Contenido secundario (nav, sidebar) vive fuera de la pantalla en mobile y aparece por gesto.

Apps con navegación lateral.

Touch, hide/show y contexto

Adaptación más allá del tamaño

Touch targets y mobile UX

El dedo no tiene la precisión de un cursor. Diseñar para touch requiere más espacio.



Tamaños mínimos

44 × 44 px Apple HIG (iOS)

48 × 48 dp Material Design (Android)

≥ 8 px Separación entre targets



Hover no existe en touch

No esconder funcionalidad detrás de hover en una UI que va a tocar un dedo.

Tooltips, menús desplegables, acciones contextuales — necesitan equivalente táctil.

Patrón: primer tap muestra hover state, segundo tap activa.



Una interfaz hecha solo con mouse en mente **se rompe en touch** incluso si el layout responde correctamente.

Hide/show responsable

Esconder contenido en mobile es válido — si lo escondido es realmente secundario.

✓ OK esconder

- ✓ Navegación secundaria (off-canvas)
- ✓ Imágenes decorativas, ilustraciones
- ✓ Filtros avanzados (detrás de un drawer)
- ✓ Promociones cross-sell

✗ NO esconder

- ✗ CTAs principales (Comprar, Contactar)
- ✗ Precios en e-commerce
- ✗ Información de contacto
- ✗ Search bar (recall vs recognition)

“ Conexión con clase 10

Recordar el information scent: si el usuario no ve una pista del contenido oculto, no sabe que existe. Esconder ≠ destruir, pero esconder mal sí destruye.

El antipatrón: Content Dispersion

El error más común al hacer responsive mal

¿Qué es content dispersion?

Investigación de NN/g (2024): un patrón generalizado en sitios modernos.

“ Content Dispersion

Páginas responsive que se renderizan en desktop con contenido "overly large and stretched out" — elementos enormes, espacios vacíos exagerados, scroll infinito para acceder a información que cabría en una pantalla.

El síntoma

En mobile la página se ve compacta y ordenada.

En desktop, los mismos componentes se inflan y dejan grandes vacíos.

El usuario tiene que hacer scroll repetidamente para ver poco contenido.

La causa

Mobile-first interpretado como "diseño en mobile y agrando".

El layout pequeño se estira sin repensar la jerarquía para una pantalla grande.

Volvemos a la advertencia del slide 7.

Los 3 culpables

Tres tendencias de diseño que combinadas producen content dispersion.



1. Mobile-first mal aplicado

Diseñar para mobile y luego solo escalar a desktop, sin reorganizar nada.

El layout de una columna se queda en una columna, solo más ancha.



2. Minimalismo excesivo

"Clean design" como sinónimo de quitar contenido. Negative space sobreaplicado.

El minimalismo era para reducir ruido, no para dejar la pantalla vacía.



3. Imágenes gigantes

Hero images decorativos del tamaño del viewport entero. Bandwidth ya no es excusa.

Empujan todo el contenido real fuera de la primera pantalla.

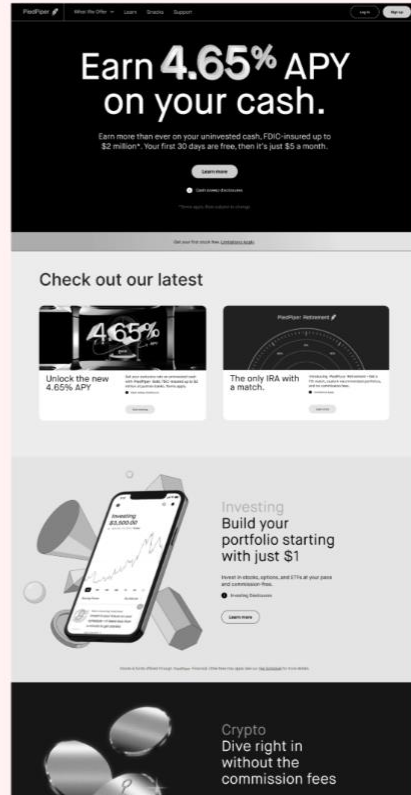


Los tres por separado son inocuos. **Combinados** producen homepages donde el usuario no encuentra nada sin scroll — ni siquiera el nombre del producto.

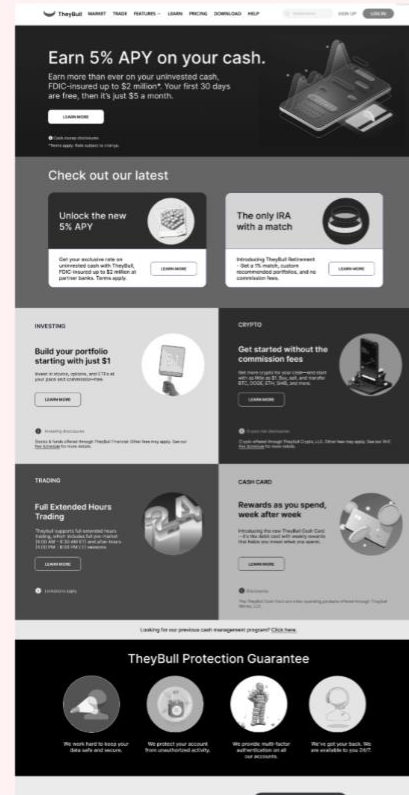
Content dispersion — ejemplo visual

Una página dispersed real. Scrollea dentro del marco para ver la altura completa.

Homepage - Dispersed Layout



Homepage - Condensed Layout



Los 4 costos de la dispersión

NN/g documentó cuatro consecuencias concretas en sus pruebas con usuarios.

1. Interaction cost ↑

Páginas largas → scroll excesivo.

Patrones mobile (accordions) traídos a desktop → clics innecesarios.

Un participante: "tengo que hacer scroll cinco segundos para volver a comparar".

2. Cognitive load ↑

Información relacionada **separada** en distintos viewports.

El usuario debe recordar lo que vio antes para comparar.

Especialmente costoso en e-commerce y comparación de planes.

3. Mental models fragmentados

El usuario no logra formarse una idea holística del producto.

Decisiones de compra se debilitan: precio, reviews y detalles aparecen separados.

Recordar también la "ilusión de completitud" — el usuario cree que la página terminó cuando no terminó.

4. Trust ↓

Layouts dispersos parecen esconder información.

Los usuarios sospechan que la empresa oculta detalles importantes.

Lo opuesto del valor declarado de "transparencia".

Recomendaciones de NN/g

Cinco principios accionables para evitar content dispersion sin perder los beneficios de mobile-first.

1. Evaluar el rendering desktop

No asumir que "se ve bien en mobile, así que está OK".
Probar explícitamente en pantalla grande.

2. Evaluar el valor de cada imagen

¿Esta imagen aporta información, o solo decora? Si decora,
¿por qué ocupa 80% del viewport?

3. Agrupar contenido relacionado

Información que el usuario necesita simultáneamente debe estar en el mismo viewport. Precio + reviews + CTA juntos.

4. No traer interacciones mobile a desktop

Accordions y drawers tienen sentido en mobile por restricción de espacio. En desktop, expandir el contenido.

5. Dispersar solo cuando se justifica

Dispersión tiene casos válidos: contenido focal complejo (un paso a la vez), imagery editorial de alto valor (productos, fotografía).

La excepción, no la regla.

“ La síntesis

Responsive desktop ≠ mobile escalado. Es un rediseño del layout para aprovechar el espacio disponible.

Resumen — 6 principios



Responsive ≠ solo viewport Adaptación en 3 dimensiones: tamaño, input (touch/mouse) y capacidades del usuario (dark mode, reduced motion).



Mobile-first es priorización, no escalamiento La pantalla pequeña fuerza a decidir qué importa. El layout grande se rediseña, no se agranda.



Breakpoints por contenido Donde el layout se rompe, no donde está un iPhone. Y considerar fluid units (clamp) para una progresión continua.



Container queries para componentes Un componente que se adapta a su contenedor escala mejor en design systems que uno que solo conoce el viewport.



Touch y accesibilidad son responsive Targets de 44×44 px, no esconder funcionalidad detrás de hover, respetar prefers-reduced-motion y prefers-color-scheme.



Cuidado con content dispersion El error más común: mobile-first mal aplicado + minimalismo + imágenes gigantes = pantallas vacías y scroll infinito. Probar en desktop, no asumir.

¿Preguntas?

Clase 16 – Responsive Design



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

IIC2182 – Primer Semestre 2026